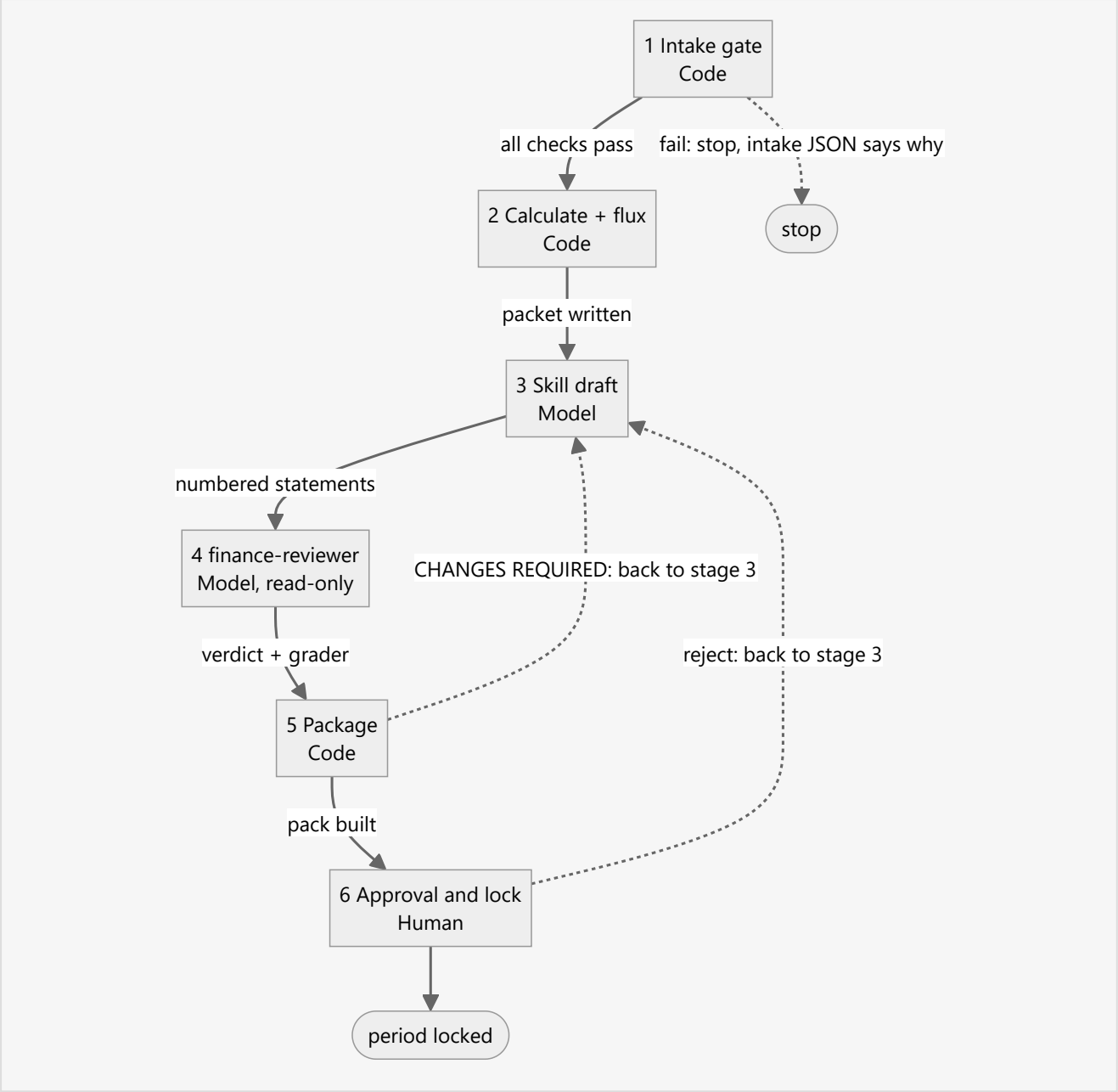
Appendix — supplementary to the one-page note

# Monthly close workflow

Six stages, one client and one month. Each stage has a gate: the next stage does not start, or its result is not accepted, until the gate passes. **Code** is deterministic Python; **Model** is Claude; **Human** is the reviewer. All files are written under `outputs/<client>/`.



| # | Stage | Who acts | Gate (what must be true) | Files produced |
|---|---|---|---|---|
| 1 | **Intake gate** | Code: `scripts/run_pipeline.py <client> <YYYY-MM>` (runs stages 1–2) | Workbook hash recorded. Income statement sheet and header row found, month columns contiguous, all six total rows present, gross profit = revenue − COGS and operating income = gross profit − opex for the latest month. Any failed check stops the run before any calculation. A period that is already approved is refused. | `intake_<month>.json` (workbook SHA-256, per-check result with cell reference), `tables_<month>.json` |
| 2 | **Calculate + flux** | Code: `src/monthly_review.py prepare`, then the flux table | Runs inside the client sandbox (no reads of other clients' files; writes only to its own output folder). Months without enough history return "not available", not zero. | `analysis.xlsx`, `summary.json`, `review_packet_<month>.json` (the only data the model sees), `flux_<month>.json`, `manifest_<run>-<client>-<month>.json` |
| 3 | **Skill draft** | Model: `/monthly-analysis` with the `monthly-financial-analysis` skill | Uses only numbers in the packet. One numbered statement per line. No invented causes. The deterministic grader (`03_evals/grade_commentary.py`) must later match every number to the packet. | `commentary_<month>.md` |
| 4 | **finance-reviewer** | Model: read-only subagent `.claude/agents/finance-reviewer.md`, given only the packet and the commentary | Returns `approve` or a numbered list of changes: wrong numbers or direction words, unsupported claims, missed movements, invented causes. It cannot edit the commentary. | `review_<month>.json` |
| 5 | **Package** | Code: `src/monthly_review.py build` | Verdict is APPROVED only if the reviewer said `approve` **and** the grader passed (every number matches, all top-3 movements named, cash risk covered when the client has cash data). Otherwise CHANGES REQUIRED. | `monthly_review_<month>.xlsx` (summary, commentary sign-off with empty sign-off column, numbers check, top movements, reviewer changes, open questions) |
| 6 | **Human approval / lock** | Human: `python scripts/approve.py <client> <month> --reviewer NAME --reason TEXT` (or the buttons in `05_review_workspace/`) | Reviewer name and reason are required. Approve is refused unless the finance-reviewer verdict is `approve`. Once approved the period is locked: no second decision, and the pipeline will not re-run it. Reject is always allowed. | `approval_<month>.json`, appended line in `audit_log.jsonl` |

## Notes

- Stages 1–2 are one command. Stages 3–5 run inside `/monthly-analysis`. Stage 6 is deliberately separate so a person, not a model, makes the final call.
- The pipeline also re-grades an existing commentary draft against a fresh packet and records a diff against the previous run in the manifest, so a re-run shows what moved.

- The source workbook is never written. QuickBooks is not part of this flow; the analysis reads the supplied workbook (see `01_connection/connection_decision.md`).
- Limits: the reviewer is an LLM and varies between runs; the deterministic checks do not judge causes or whether a movement is worth mentioning. `04_architecture/architecture.md` lists what breaks first.

# Workflows

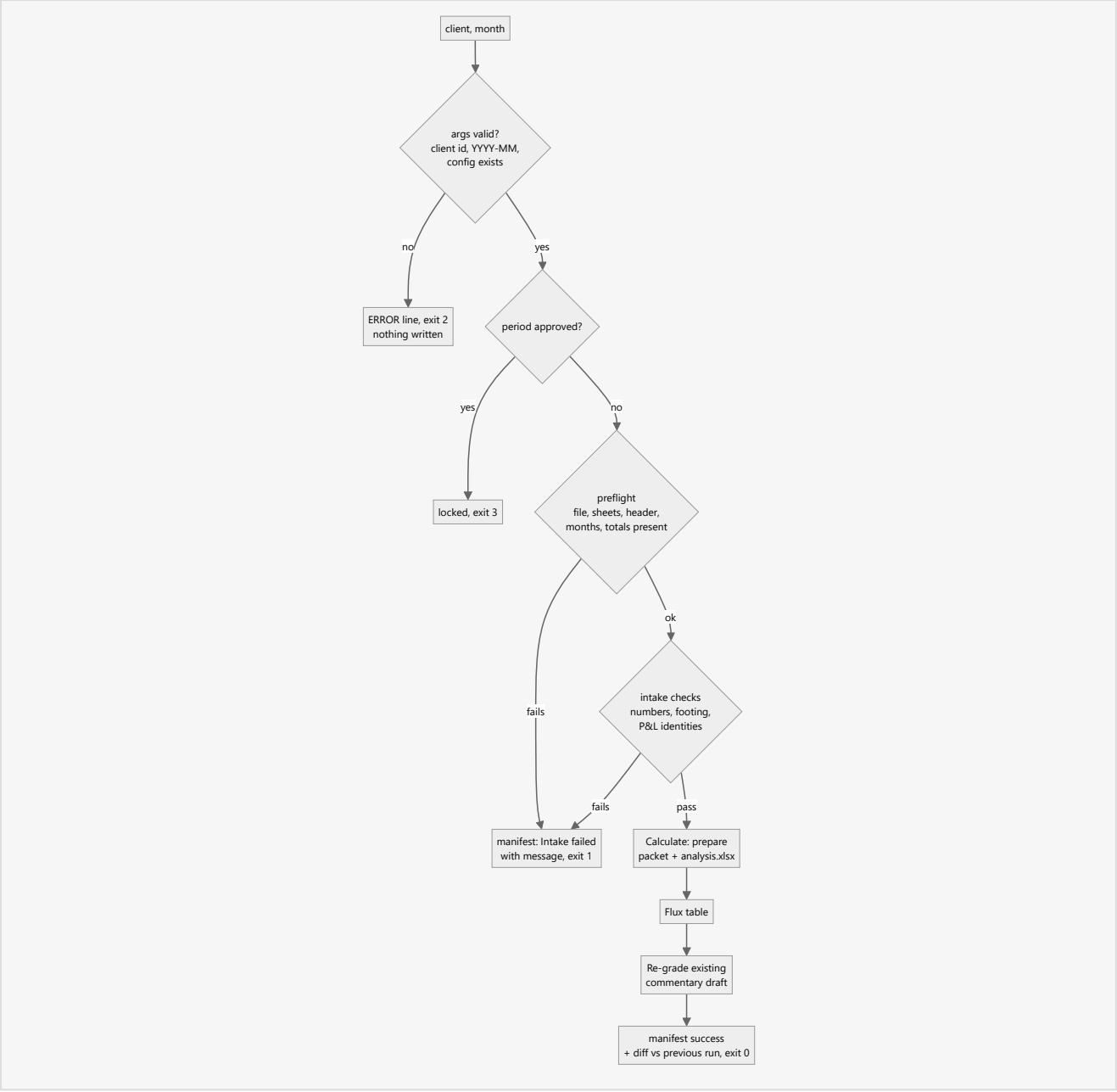
One page per workflow. Each covers inputs, outputs, gates, failure modes and what the user sees, idempotency, and the tests. Messages quoted in the failure tables come from runs on 2026-10-14.

| Workflow | Page | Top failure modes (all give one clear line, no traceback) | Tests |
|---|---|---|---|
| (a) Intake → calculate → flux | a_intake_calculate_flux.md | invalid client id / path traversal; unknown client or bad month; workbook refused by intake | `test_workflow_failures.Pipeline`, `test_robustness`, fault suite |
| (b) `/monthly-analysis` | b_monthly_analysis_skill.md | intake refusal in `prepare`; missing or invalid packet, commentary or verdict in `build`; unnumbered commentary | `test_workflow_failures.SkillSteps`, `test_monthly_review` |
| (c) Approve / reject / lock | c_approve_reject_lock.md | locked period; damaged decision file; verdict not approve | `test_workflow_failures.Approval`, `test_monthly_review` |
| (d) QBO test and evidence | d_qbo_connection_and_evidence.md | server missing; token or keys invalid; invalid client id | `test_workflow_failures.Connection`, `test_qbo_diagnose` |
| (e) Loader dry-run / compare | e_sandbox_loader_dry_run.md | bad month range; empty range; missing or invalid compare file | `test_workflow_failures.Loader`, `test_qbo_load` |
| (f) Review workspace UI | f_review_workspace_ui.md | missing outputs; unreadable JSON; failed or hung script | `test_ui_numbers`, `test_workflow_failures.WorkspaceActions` |
| (g) Package submission | g_package_submission.md | secret finding; output outside repo; no git | `test_workflow_failures.Packaging`, `test_scan_secrets` |

Known gap: there is no "re-run as v2" after approval. See (c) for why this was left as the owner's decision.

# (a) Intake → Calculate → Flux

`python scripts/run_pipeline.py <client> <YYYY-MM> [--clients-root DIR --outputs-root DIR]`. It runs the deterministic stages of a close, and the review workspace's **Run analysis** button runs the same command.




## Inputs

- `clients/<client>/config.yaml` and the workbook it names (read-only).
- The month to analyse, which must have its prior month in the data.

## Outputs (all under `outputs/<client>/`)

- `intake_<m>.json` and `tables_<m>.json`;
- `review_packet_<m>.json`, `analysis.xlsx`, `summary.json` and `commentary.md`;
- `flux_<m>.json`;

- `commentary_<m>.json`, only if a draft exists;
- `manifest_<run_id>.json`;
- one line appended to `audit_log.jsonl`.

## Gates

1. **Argument check, before any path is built.** The client id must match `^[a-z0-9][a-z0-9_-]{0,63}$`, the month must be `YYYY-MM`, and the client's config must exist.
2. **Lock.** `approval_<m>.json` with status `approved` stops the run.
3. **Intake gate** (`close_artifacts.intake_gate`, the same function `/monthly-analysis` uses): - *Preflight:* the file is readable, not locked and a real .xlsx. The required sheets are present. Formula cells have cached values, and no cell holds an Excel error value. The header row, unmerged month headers, no duplicate months, the requested month and its prior month, and every total and section-total row must all be present. - *Checks:* every amount is a number; month columns are contiguous; gross profit and operating income tie out in every month; each section's lines foot to its total in every month.
4. **Client sandbox.** Reads are limited to the client's own folders and writes to `outputs/<client>/`.

## Failure modes and what the user sees (from runs on 2026-10-14)

| Failure | Exit | Message (abridged) |
|---|---|---|
| Client id like `../x` | 2 | `ERROR: invalid client id '../x'; use lowercase letters, digits, '-' or '_' (e.g. abc).` Before the fix this crashed with a traceback while trying to write `outputs/../x/`. |
| Unknown client | 2 | `ERROR: unknown client 'nope': no clients\nope\config.yaml. Add a config (see 02_skill/README.md).` |
| Bad month `2026-13` | 2 | `ERROR: month must look like 2026-06, got '2026-13'.` Before: a raw `DateParseError` |
| Month not in the data | 1 | `Intake: failed - Intake refused the workbook: Income Statement: no column for 2024-01 (the requested month) …` |
| Workbook problem (32 kinds) | 1 | The intake message names the cell or sheet, e.g. `Total Income foots to its lines (every month) at Income Statement!F14 (2026-03: lines - total = 1,000.00)`. See `03_evals/robustness/ROBUSTNESS_REPORT.md` |
| Period approved | 3 | `abc 2026-06 is approved and locked; not re-running.` |

## Idempotency

- A re-run with the same workbook rewrites the same files with the same numbers.
- Each run adds a new manifest whose `diff_vs_previous.changed` is empty if nothing changed, plus one audit line.
- A failed run writes a manifest and an audit line, but no packet.

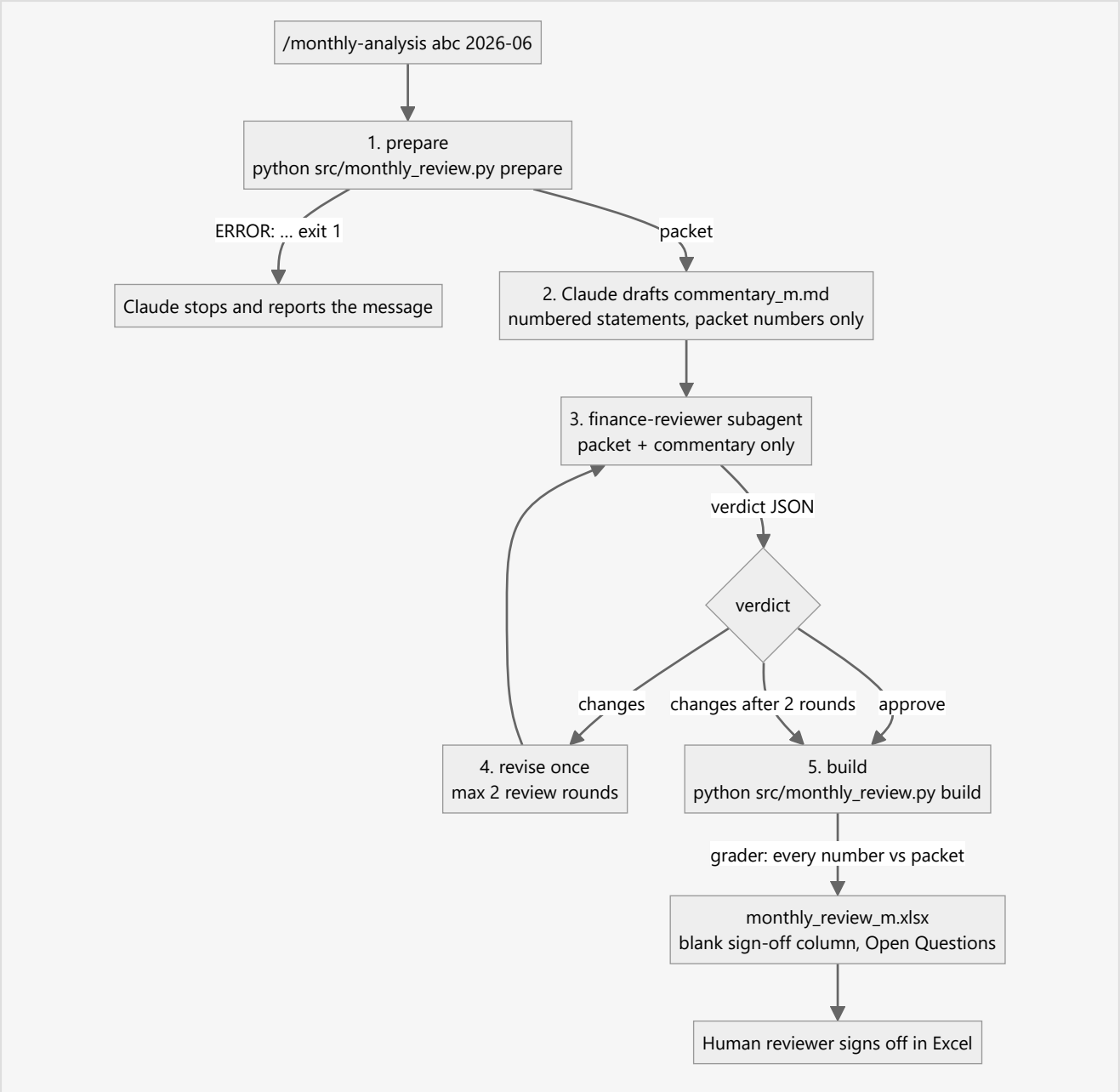
## Tested in

- `tests/test_workflow_failures.py`: `Pipeline.*` covers traversal, bad month, unknown client and the sandbox id check.
- `tests/test_robustness.py`: the intake gate.
- `03_evals/robustness/fault_injection.py`: 32 faults.

- `tests/test_client_isolation.py`.
- `tests/test_ui_numbers.py`, which reads pipeline outputs.
- Golden G19–G22.

# (b) `/monthly-analysis`: draft → finance-reviewer → grade → pack

A Claude Code command (`.claude/commands/monthly-analysis.md`). Code computes every number; Claude writes and reviews the text; a person signs off.




## Inputs

- Client and month.
- The workbook, through the client config.
- The `monthly-financial-analysis` skill guidance.
- The finance-reviewer agent (`.claude/agents/`), which is read-only.

## Outputs (`outputs/<client>/`)

- From `prepare`: `analysis.xlsx` and `review_packet_<m>.json`.
- From Claude: `commentary_<m>.md`.
- From the reviewer: `review_<m>.json`.
- From `build`: `monthly_review_<m>.xlsx`.

## Gates

1. `prepare` **runs the same intake gate as the pipeline**, before anything is written. Until Phase 2 this path skipped intake, and 8 of 32 corrupted workbooks produced wrong numbers silently.
2. The commentary must be numbered `1.`, `2.`, … in order (`parse_statements`).
3. Every number in each statement is graded against the packet (`grade_commentary.grade`). A mismatch, a reviewer "changes" verdict, or missing cash coverage blocks approval in the pack.
4. Sign-off cells are always blank; nothing is signed for anyone.

## Failure modes and what the user sees (from runs on 2026-10-14)

| Failure | Exit | Message |
|---|---|---|
| Workbook refused by intake | 1 | `ERROR: Intake refused the workbook: …` or `ERROR: Intake checks failed …`, naming the cell. Before Phase 3 this was a full Python traceback. |
| Unknown client or bad month | 1 | `ERROR: Unknown client 'nope': …config.yaml not found. Add a client config (see 02_skill/README.md).` and `ERROR: Month must look like 2026-06, got '2026-13'` |
| `build` without a packet, commentary or verdict, or with an invalid verdict JSON | 1 | `ERROR: Review packet not found: … Run 'python src/monthly_review.py prepare abc 2026-06' first.` and `ERROR: Reviewer verdict is not valid JSON: … (line 1)` |
| Commentary not numbered | 1 | `ERROR: Commentary must contain statements numbered 1., 2., 3. ... in order, one per line.` |
| Claude invents or miscopies a number | 0 | The pack's number-check column shows `MISMATCH: …` and approval is blocked (`test_wrong_number_blocks_approval_even_if_reviewer_approves`). |

## Idempotency

- `prepare` is deterministic: the same packet every time.
- Drafting is not deterministic, because the model writes the text. The grader judges each draft against the packet, never against an earlier draft.
- `build` always writes `monthly_review_<m>.xlsx`; a second build overwrites the first. Keep a control copy under another name, as `_flawed_control.xlsx` is kept.

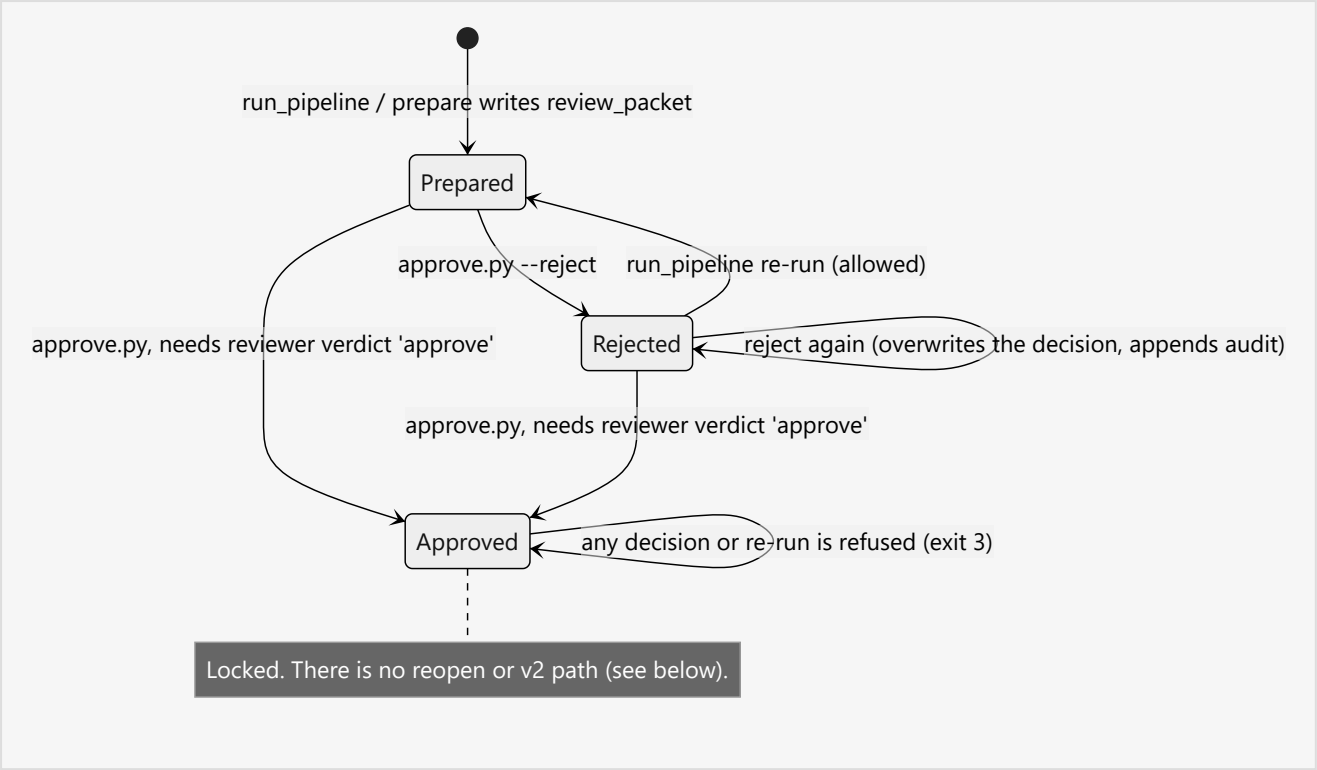
## Tested in

- `tests/test_workflow_failures.py`: `SkillSteps.*`.
- `tests/test_monthly_review.py`: numbering, mismatch blocks approval, reviewer changes block approval, missing cash blocks approval.
- `tests/test_robustness.py`: `SkillPathUsesTheGate`.
- `03_evals/grade_commentary.py`, with a flawed control that must FAIL.

- `fault_injection.py`, which runs every fault through `prepare`.
- Not tested: a live `/monthly-analysis` run in Claude. The saved skill output is graded as it is on disk.

# (c) Approve / reject / lock / re-run

`python scripts/approve.py <client> <YYYY-MM> --reviewer NAME --reason TEXT [--reject]`. The workspace's Approve and Reject buttons run the same command.




## Inputs

- `outputs/<client>/review_packet_<m>.json`, which is required.
- `review_<m>.json` (the finance-reviewer verdict), which approval requires.
- A reviewer name and a reason, both non-empty.

## Outputs

- `approval_<m>.json`, recording the status, reviewer, reason, time, verdict and the packet's SHA-256.
- One `period_approved` or `period_rejected` line appended to `audit_log.jsonl`.

## Gates

1. Client id and period format are checked.
2. The reviewer name and reason must be non-empty.
3. The packet must exist.
4. An already-approved period is locked.
5. Approving needs the reviewer's verdict to be `approve`.
6. A decision or verdict file that can't be read refuses the request and records nothing. It never guesses.

## Failure modes and what the user sees (from runs and tests, 2026-10-14)

| Failure | Exit | Message |
|---|---|---|
| Approved period | 3 | `ERROR: abc 2026-06 is approved and locked`. The pipeline also refuses: `… is approved and locked; not re-running.` |
| No packet | 1 | `ERROR: no review packet for zz 2026-06; run the pipeline first` |
| Reviewer verdict is not `approve` | 4 | `ERROR: cannot approve: finance-reviewer verdict is 'changes', not 'approve'` |
| Missing name or reason; bad period | 2 | `ERROR: reviewer name and reason must not be empty` and `ERROR: invalid client or period` |
| Damaged `approval_<m>.json` or `review_<m>.json` | 5 | `ERROR: cannot read an existing decision or verdict file as JSON (JSONDecodeError); check … Nothing was recorded.` Before Phase 3: a traceback |

## Idempotency

- Repeating a reject overwrites `approval_<m>.json`, and each attempt adds an audit line, so history is kept in the log, not in the file.
- Approval is final.
- The packet hash in the approval record shows which numbers were approved.

## Re-run as v2: not implemented

The brief workflow asks for "re-run as v2" after approval. Today an approved period can't be re-run, reopened or versioned. The pipeline exits 3, and `approve.py` refuses. Deleting `approval_<m>.json` by hand would bypass the lock, and the audit log would still show the approval.

A v2 path needs a controls decision first: - who may reopen an approved period; - whether v1's packet and pack are archived (for example `*_v1.*`) before v2 is written; - how the reopen is recorded in the audit log.

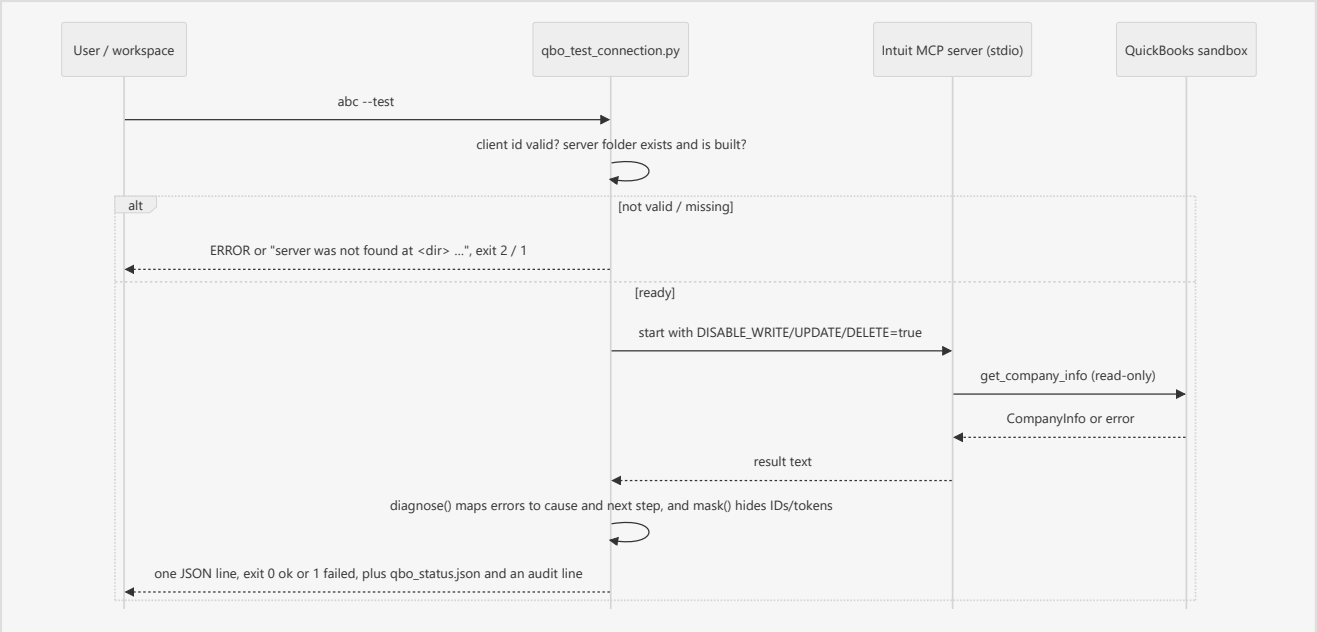
That decision is the owner's, so it was not built.

## Tested in

- `tests/test_workflow_failures.py`: `Approval.*` covers the damaged file (exit 5, no audit line) and the lock (exit 3).
- `tests/test_ui_numbers.py`: `test_approved_period_renders_as_locked`.
- `tests/test_monthly_review.py`: the reviewer verdict blocks approval.

# (d) QuickBooks connection test and evidence capture

- `python 01_connection/qbo_test_connection.py <client> [--test]` refreshes the status and, with `--test`, makes one read-only CompanyInfo call. The workspace's **Test connection** button runs it.
- `python 01_connection/evidence/quickbooks/capture_dated.py` saves today's redacted, read-only checks.




## Inputs

- The server checkout: `quickbooks-online-mcp-server/` beside the repo, or the folder named by `QBO_MCP_SERVER_DIR`.
- The server's own environment file, which this repo never reads.
- Node.js on PATH.

## Outputs

- `qbo_test_connection.py` writes `outputs/<client>/qbo_status.json` (flags, last test, last evidence date) and appends an audit line.
- `capture_dated.py` writes `01_connection/evidence/quickbooks/<YYYY-MM-DD>/` (redacted requests and responses, `summary.json`).

## Gates

1. The client id is validated.
2. The server must exist and be built.
3. The write, update and delete disable flags are forced to `true` on every start. The server then doesn't register any create, update or delete tool (`src/helpers/register-tool.ts`), and the 2026-10-14 capture recorded 71 tools with none of them a write tool.
4. Server text is stored only after `mask()` (IDs and tokens removed) or `redact()`.
5. The capture exits 1 if a secret-like pattern survives redaction.

## Failure modes and what the user sees

| Failure | Exit | Message |
|---|---|---|
| Server not cloned or built | 1 | `The QuickBooks MCP server was not found at '<dir>'. It is not part of this repo: clone Intuit's quickbooks-online-mcp-server there (or set QBO_MCP_SERVER_DIR …), then run 'npm install', 'npm run build' and 'npm run auth' in it.` Before Phase 1 this exited 0. |
| Expired or revoked token; wrong keys; wrong environment | 1 | A fixed next step per cause, e.g. `The refresh token is expired or revoked. In quickbooks-online-mcp-server run 'npm run auth' …`. Seven causes, in `DIAGNOSES` |
| Network or outage | 1 | `The server could not reach quickbooks.api.intuit.com. Check internet access, VPN or proxy, then retest.` or `No response within 45 seconds …` |
| Invalid client id | 2 | `ERROR: invalid client id 'ABC'; use lowercase letters, digits, '-' or '_' (e.g. abc).` Before Phase 3 this exited 2 with no output. |
| Node.js missing | 1 | `Node.js was not found. Install Node.js and make sure 'node' is on PATH.` |

## Idempotency

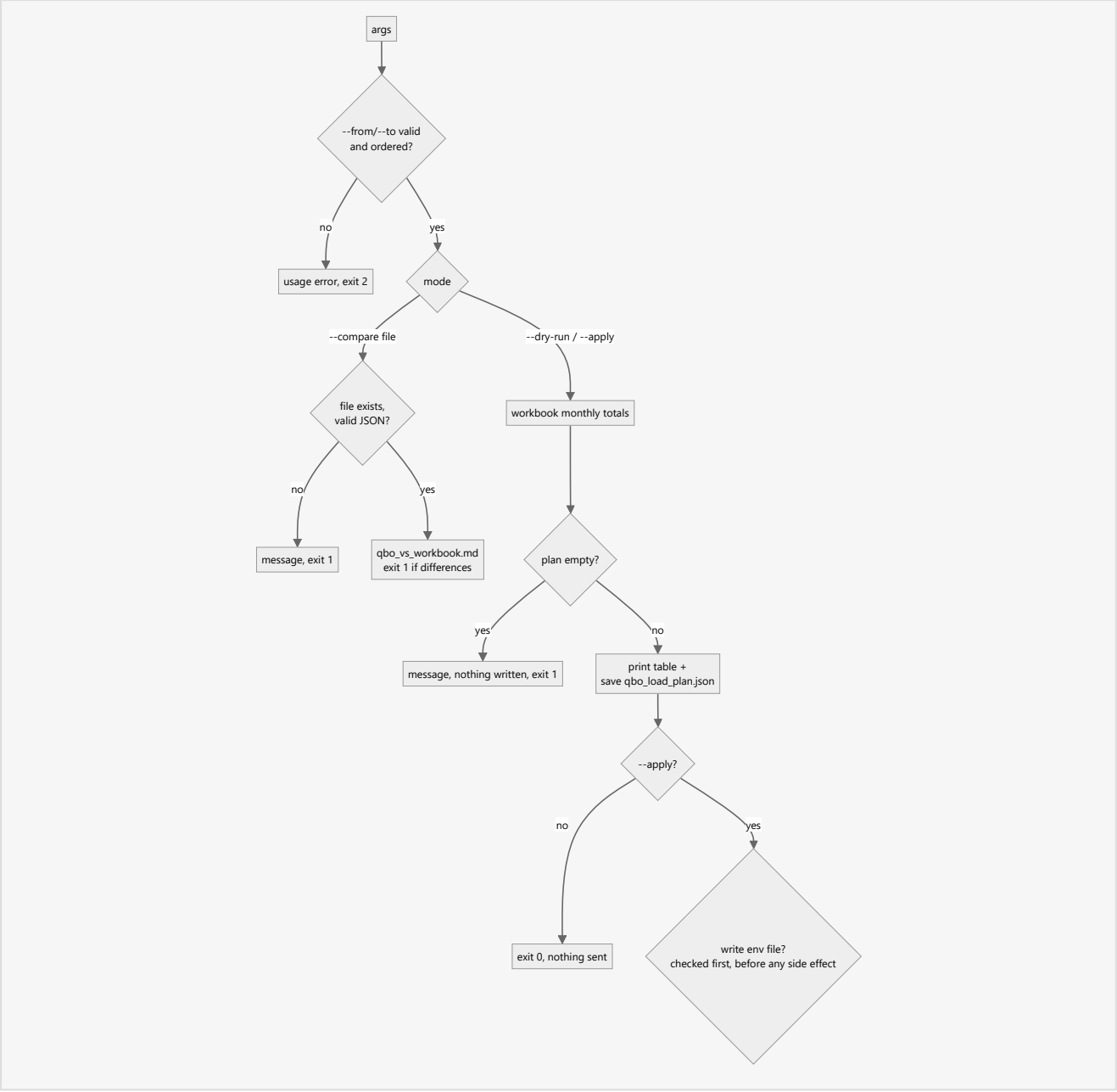
- Each `--test` overwrites `qbo_status.json` and adds one audit line; a failed test is recorded as `failed`.
- A capture overwrites that day's folder.
- Nothing is written to QuickBooks: no write tool exists in the session.

## Tested in

- `tests/test_workflow_failures.py`: `Connection.*` covers the invalid client, and the missing server (exit 1, status written to a temp root).
- `tests/test_qbo_diagnose.py`: every cause has a next step; stored text hides IDs.
- `tests/test_evidence_redaction.py`.
- Live: `01_connection/evidence/quickbooks/2026-10-14/README.md` (checks a–d).

# (e) Sandbox loader `--dry-run`, and `--compare`

`python 01_connection/load_workbook_to_sandbox.py [--dry-run | --apply | --compare PNL_JSON] [--from YYYY-MM] [--to YYYY-MM]`. The default is `--dry-run`. `--apply` has never been run against the sandbox (stretch goal not done).




## Inputs

- The ABC workbook, through `clients/abc/config.yaml`.
- An optional month range.
- For `--compare`: a saved QBO monthly P&L, such as the one `capture_dated.py` writes.
- For `--apply`: a write-scoped env file outside the repo (`.secrets/`). The repo never reads or prints it.

## Outputs

- `outputs/abc/qbo_load_plan.json`, which is the plan for exactly the selected range.
- `outputs/abc/qbo_vs_workbook.md`, from `--compare`.

## Gates

- Month arguments are validated.
- An empty plan is never saved.
- `--apply` without its env file stops before any file or network side effect.
- Every journal entry must balance (debits = credits) before it is planned.

## Failure modes and what the user sees (from runs on 2026-10-14)

| Failure | Exit | Message |
|---|---|---|
| Bad `--from`/`--to` | 2 | `error: --from must look like 2026-06, got '2026-13'` and `error: --from 2026-06 is after --to 2026-01`. Before Phase 3: a `DateParseError` traceback |
| Range with no workbook months | 1 | `No workbook months between 2027-01 and the end (workbook: 2025-01 to 2026-06); nothing was written.` Before Phase 3 this exited 0 and **overwrote the saved plan with an empty one**. |
| `--compare` file missing or not JSON | 1 | `--compare file not found: … Save a QBO monthly P&L first (… capture_dated.py).` and `--compare file is not valid JSON: …`. Before Phase 3: tracebacks |
| `--apply` without the env file | 1 | `--apply needs the write-scoped env file: … (not found; nothing was changed)` |
| `--compare` finds no loaded accounts (today's sandbox) | 1 | A table row per metric: `account 'Workbook Load - Total Income' not in the QBO P&L` |

## Idempotency

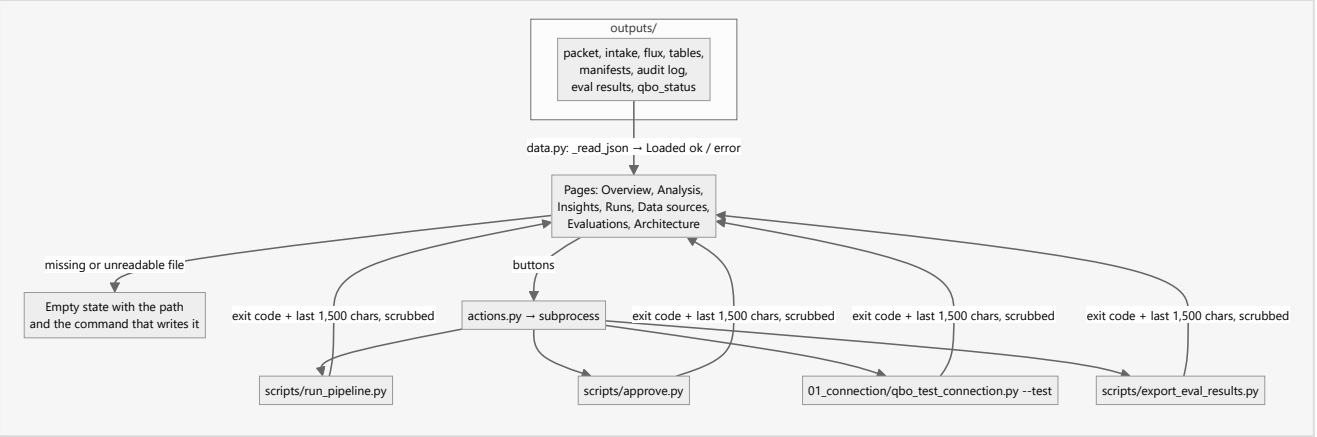
- A dry run is deterministic. A dry run with a range replaces the saved plan with that range's plan, which is what `--apply` would create.
- `--apply` skips entries that already exist with the same amounts, and reports a different existing entry as a CONFLICT, leaving it alone (tested with an in-memory fake, never live).

## Tested in

- `tests/test_workflow_failures.py`: `Loader.*` covers bad dates, reversed range, bad compare file, and an empty range writing nothing.
- `tests/test_qbo_load.py`: dry run creates nothing; apply is idempotent; conflict left alone; apply without env file stops before network and writes nothing.

# (f) Review workspace UI (display only)

`streamlit run 05_review_workspace/app/Home.py`. The UI only reads JSON and Excel that the engine wrote under `outputs/`, and it never calculates a figure (`CLAUDE.md`, "UI rule"). Its buttons run the same scripts as the command line.




## Inputs

- JSON and Excel in `outputs/` (see workflows a–d).
- The client and period chosen in the sidebar.

## Outputs

- None of its own. Buttons cause the scripts above to write their usual outputs.

## Gates

1. **No calculation modules:** pages may not import the engine, pandas maths helpers, etc. (an AST check, plus a clean-interpreter import check).
2. **Numbers come from the engine:** every amount and percentage displayed must exist in the engine JSON.
3. **Secrets are withheld:** credential-like fields in a JSON file are not shown, and script output is passed through `ui.scrub` before display.
4. **Locked periods:** an approved period disables Run analysis (`runs.py`) and hides the decision form (`insights.py`).

## Failure modes and what the user sees

| Failure | What is shown |
|---|---|
| No clients, or a client with no engine files | An empty state naming the missing file and the command that writes it (`Reviewer Excel not available yet: outputs/<client>/monthly_review_<period>.xlsx`) |
| Unreadable or corrupt JSON | An empty state with the file path; never a traceback (`test_unreadable_json_shows_an_empty_state_not_a_traceback`) |
| A button's script fails | `Pipeline failed.`, `Golden evals did not complete.`, or the script's own `ERROR:` line, scrubbed. Since Phases 1 and 3, the scripts exit non-zero on failure, so the UI no longer shows a failed connection test as success. |

| Failure | What is shown |
|---|---|
| A button's script hangs | `scripts/run_pipeline.py did not finish within 180 seconds and was stopped. Run it from a terminal to see its progress.` Before Phase 3: "Could not start script: TimeoutExpired" |
| Python or Node missing | `Could not start <script>: FileNotFoundError` and, on Data sources, `The test script could not run. Check that Python and Node.js are installed …` |

## Idempotency

- Pages are read-only and render the same files the same way.
- Buttons are as idempotent as the scripts they run (see a, c, d).

## Tested in

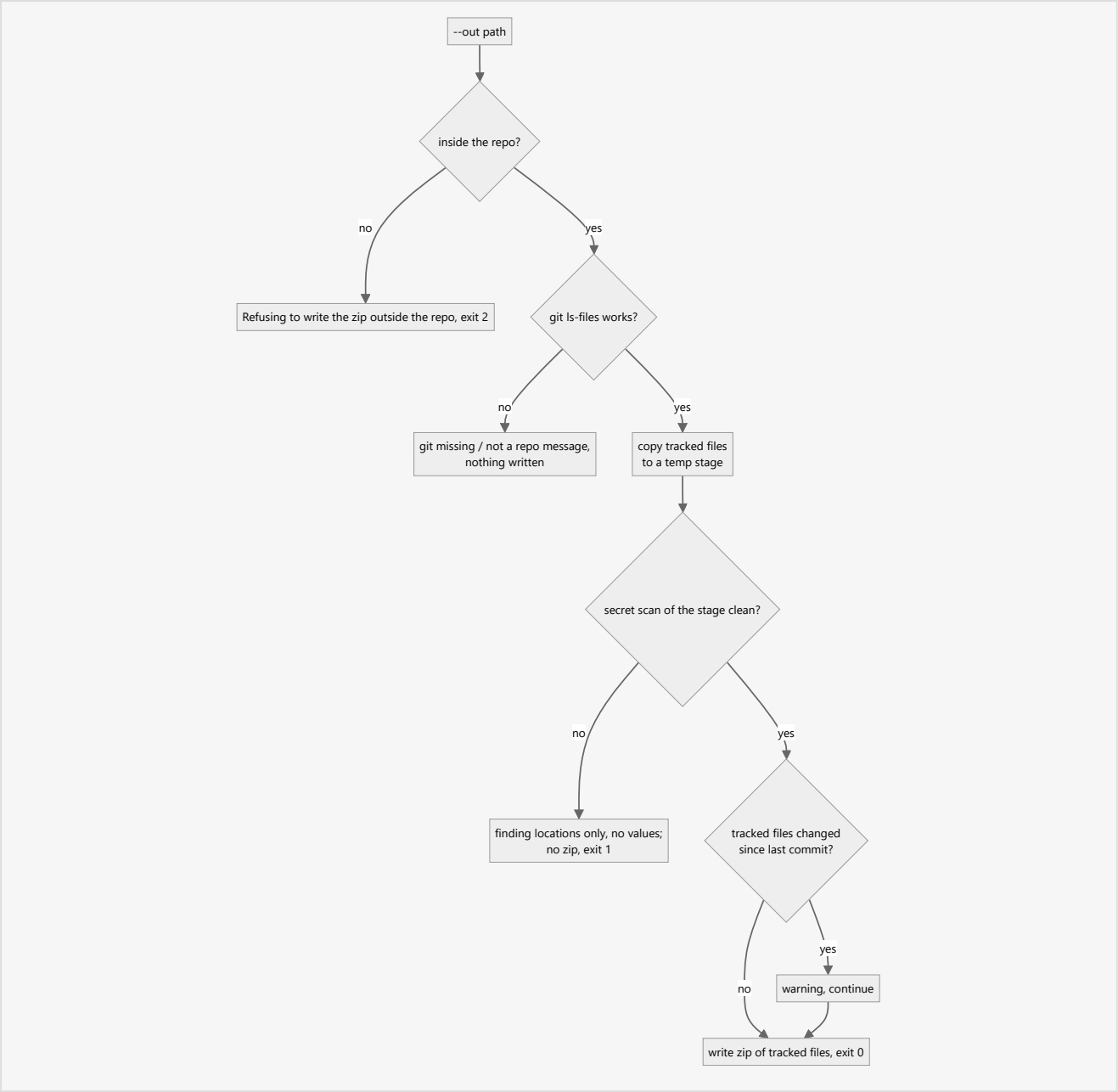
`tests/test_ui_numbers.py` (15 tests): - every page renders without exceptions; - displayed numbers exist in the engine JSON; - no secrets are shown; - empty states; - unreadable JSON; - credentials withheld; - approved period locked.

`tests/test_workflow_failures.py`: `WorkspaceActions` (timeout message).

Not tested: the buttons are not clicked against a running Streamlit server, and the browser rendering is not checked.

# (g) Package the submission

`python scripts/package_submission.py [--out dist/submission.zip]`




## Inputs

- The git-tracked files only. Ignored files (environment files, tokens, `node_modules`, the untracked MCP server clone) can't get in.

## Outputs

- `dist/submission.zip`, or the `--out` path inside the repo.

## Gates

1. The output path must be inside the repo.
2. git must be available.

3. The secret scan runs on a staged copy of exactly the files that ship. Any finding stops the build, and the scan prints file:line and pattern name, never the value.
4. A dirty working tree gives a warning.

## Failure modes and what the user sees

| Failure | Exit | Message |
|---|---|---|
| Secret-like value in a tracked file | 1 | `Secret-like findings: N` with file:line and kind, then `Secret scan is not clean; no zip was built.` |
| `--out` outside the repo | 2 | `Refusing to write the zip outside the repo.` |
| git missing or not a repository | 1 | `Cannot list tracked files: git is not installed or this folder is not a git repository. … nothing was written.` Before Phase 3: a `CalledProcessError` or `FileNotFoundError` traceback |
| Uncommitted changes | 0 | `Warning: tracked files have uncommitted changes; the zip holds the working-tree versions.` |

## Idempotency

- The previous zip is deleted and rebuilt from the same tracked-file list.
- Contents equal the working tree at the time of the run.

## Tested in

- `tests/test_workflow_failures.py`: `Packaging.*` covers no git (no zip written) and a zip outside the repo.
- `tests/test_scan_secrets.py`: planted values found, values never printed, file paths scanned, missing path fails.
- `tests/test_hooks.py`: secret files blocked from tool reads.